

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



ÉTICA Y SEGURIDAD DE DATOS

PROYECTO PARCIAL – DS3031

Panel seguro para el análisis de entregas de Olist

Caso de negocio: retrasos logísticos y satisfacción del cliente

Integrantes

Ronal Jesus Condor Blas

Marco Soto Maceda

Enlaces del proyecto

Repositorio y código fuente

<https://github.com/ronalforge/proyecto-etica-seguridad-olist>

Informe publicado en GitHub Pages

<https://marcosotomac.github.io/proyecto-etica-seguridad-olist/>

Lima, Perú

Septiembre de 2026

Índice

1	Resumen ejecutivo	1
2	Introducción y caso de negocio	1
2.1	Problema y oportunidad	1
2.2	Objetivos	1
2.3	Valor esperado y criterio de éxito	2
3	Fuentes de datos, alcance y calidad	2
3.1	Dataset principal y fuente complementaria	2
3.2	Selección y limitaciones del dato	3
3.3	Definición de indicadores	3
4	Análisis descriptivo y utilidad	4
5	Requerimientos y diseño de la solución	4
5.1	Requerimientos funcionales	4
5.2	Arquitectura y flujo	5
5.3	Minimización y ciclo de vida	5
6	Diseño de seguridad y ética	5
6.1	Modelo de amenazas	5
6.2	Protección en reposo y en tránsito	6
6.3	Accesos y registros de auditoría	6
6.4	Protección de datos y marco de referencia	7
7	Estrategia de uso seguro, políticas y formación	7
7.1	Política propuesta para el equipo	7
7.2	Procedimiento y concientización	8
8	Respuesta ante incidentes y recuperación	8
8.1	Plan básico	8
8.2	Contingencia y respaldo	9
9	Implementación y validación	9
9.1	Componentes desarrollados	9
9.2	Pruebas y evidencias técnicas	9
9.3	Evidencias visuales del prototipo	10
10	Recomendaciones de protección futura	13
11	Lecciones aprendidas y retrospectiva del equipo	13
11.1	Lecciones técnicas y analíticas	13
11.2	Retrospectiva del trabajo en equipo y mejoras futuras	14
12	Conclusiones	14

1. Resumen ejecutivo

Desarrollamos un prototipo académico para analizar las entregas tardías del conjunto histórico de comercio electrónico de Olist. Buscamos identificar segmentos que merecen una investigación logística y describir cómo varía la calificación de la compra según el cumplimiento de la fecha estimada de entrega. Para ampliar el análisis, incorporamos una segunda fuente: el calendario oficial de feriados nacionales de Brasil de 2018.

De los 99 441 pedidos del archivo original, seleccionamos 96 470 entregados con fechas y estado del cliente disponibles. Entre ellos, 6 534 llegaron después de la fecha estimada: **6.8 %**. El retraso medio de esos pedidos fue de **10.6 días**. La calificación promedio, calculada con las reseñas disponibles, fue de **4.29/5** para las entregas puntuales y **2.27/5** para las tardías. La comparación es descriptiva y no demuestra causalidad.

Implementamos una base SQLite cifrada con SQLCipher, un backend Flask y un panel React con componentes shaden/ui. El sistema incluye autenticación, roles, consultas agregadas, registro de auditoría, HTTPS local con certificado autofirmado y respaldos cifrados y autenticados con AES-256-GCM. Reconocemos tres límites de la demostración: la clave de la base permanece en el mismo equipo, el ZIP original no está cifrado y el modo HTTP local **no protege el tráfico**. Además, documentamos un procedimiento de uso seguro, un plan básico ante incidentes y mejoras futuras.

2. Introducción y caso de negocio

2.1. Problema y oportunidad

En una operación de comercio electrónico, conocer solo el número total de pedidos no permite priorizar la revisión de demoras. Un indicador global puede ocultar diferencias entre estados o meses, mientras que una lista de pedidos individuales expondría datos que el analista no necesita. Se plantea una vista agregada que permita responder: ¿cuál fue la tasa de tardanza?, ¿en qué grupos históricos fue mayor?, y ¿cómo varió la calificación observada entre pedidos puntuales y tardíos?

El usuario previsto es un analista u operador que necesita orientar una investigación posterior. La aplicación no determina responsabilidades de vendedores, no predice plazos de entrega y no toma decisiones individuales sobre clientes. El dataset es histórico (compras de 2016 a 2018); no representa por sí solo el desempeño actual de Olist ni de otra tienda.

2.2. Objetivos

Objetivo general: diseñar y demostrar un sistema de análisis agregado de entregas tardías que aporte métricas reproducibles y aplique controles de seguridad proporcionales al caso académico.

Objetivos específicos:

1. Integrar el dataset Olist con una fuente complementaria verificable de feriados.

2. Definir KPIs con denominadores explícitos y explorar variaciones por estado y mes.
3. Implementar una base de datos, un backend y una interfaz de consulta con acceso restringido.
4. Demostrar controles de autenticación, auditoría, transporte HTTPS local y recuperación mediante respaldo cifrado.
5. Documentar riesgos, límites éticos, respuesta a incidentes y mejoras futuras sin atribuir al prototipo garantías que no ofrece.

2.3. Valor esperado y criterio de éxito

El valor del panel es reducir el tiempo para ubicar segmentos que conviene examinar y evitar compartir filas individuales en la exploración inicial. Se mide mediante la disponibilidad de indicadores con definición reproducible, el filtrado por estado y la capacidad de restaurar una copia íntegra. La línea de base de 6.8 % describe el dataset; **no se fija una meta ficticia de reducción** para datos históricos. Una meta operativa futura requeriría datos recientes, un período de comparación y acuerdo con el responsable del negocio.

3. Fuentes de datos, alcance y calidad

3.1. Dataset principal y fuente complementaria

El *Brazilian E-Commerce Public Dataset by Olist* reúne archivos CSV de pedidos, clientes, ítems y reseñas [2]. La copia ZIP utilizada en esta ejecución tiene SHA-256 967E41E04FC306FE604E2A693F488995A8B41E5047418F8A5C8E4ABD6DECA784. El archivo original se conserva localmente en `data/olist.zip`, fuera del repositorio Git. El origen es público, pero ello no elimina los riesgos de reidentificación o uso fuera de contexto.

La segunda fuente es la Portaria brasileña n.º 468/2017, que publica los feriados nacionales de 2018 [3]. Se transcribieron nueve fechas nacionales a `holidays_2018.csv`; se excluyeron puntos facultativos y feriados estatales o municipales. La unión se hace por *fecha de compra*, no por fecha de entrega. Esta decisión es una hipótesis exploratoria y no implica que el feriado causara un retraso.

Fuente	Uso en el proyecto	Campos relevantes
Olist: pedidos	Selección de pedidos entregados y cálculo temporal	Estado y fechas de compra, estimada y real
Olist: clientes	Agrupación geográfica amplia	Estado federado
Olist: reseñas	Satisfacción observada	Puntuación 1–5
Olist: ítems	Campo de vendedor para posible extensión; no se muestra	Identificador de vendedor
Portaria 468/2017	Etiquetar compras en feriado nacional de 2018	Fecha y nombre del feriado

Tabla 1: Procedencia y uso de las fuentes.

3.2. Selección y limitaciones del dato

El importador conserva pedidos con estado `delivered`, fechas de compra, entrega estimada y entrega real, y estado del cliente disponibles. Del total de 99 441 pedidos, 96 478 constan como entregados; ocho de ellos no cumplen todos los campos necesarios. En consecuencia, se incluyen 96 470 y se excluyen 2 971 en total. El período de compra incluido va del 15 de septiembre de 2016 al 29 de agosto de 2018; los extremos de ambos años son parciales.

En la tabla importada faltan 646 puntuaciones de reseña (0.67 % del total). La media de reseñas puntuales usa 89 443 puntuaciones disponibles entre 89 936 pedidos puntuales; la de tardías usa 6 381 puntuaciones entre 6 534 pedidos tardíos. Los pedidos sin reseña siguen en el denominador de la tasa de tardanza, pero no en el promedio de puntuación. El importador toma la primera reseña y el primer vendedor que encuentra por pedido; por ello, no se presenta una medida por vendedor ni se asume que exista una única reseña por pedido. En las 96 470 filas importadas no se observaron fechas de entrega anteriores a la compra, puntuaciones fuera de 1–5 ni incoherencias entre el indicador de tardanza y la diferencia de fechas. Estos chequeos no sustituyen una auditoría completa del dataset.

3.3. Definición de indicadores

Se convierte cada marca temporal a fecha calendario. Para el pedido i , d_i es la diferencia en días entre fecha real y estimada y $t_i = 1$ si $d_i > 0$; en otro caso $t_i = 0$. Al truncar a fecha, una entrega posterior dentro del mismo día estimado se considera puntual. Esta es una convención del prototipo que debe explicitarse al interpretar los resultados.

$$\text{Tasa tardía} = \frac{\sum_i t_i}{N} \times 100, \quad \text{Retraso medio} = \frac{\sum_{i:t_i=1} d_i}{\sum_i t_i}. \quad (1)$$

Las puntuaciones medias se calculan solo para los pedidos con `review_score` presente dentro de cada grupo. Los porcentajes se redondean a una cifra decimal para la presentación; el cálculo usa todos los registros incluidos.

Indicador	Resultado	Base de cálculo
Pedidos analizados	96 470	Entregados y completos
Entregas tardías	6 534; 6.8 %	6 534 / 96 470
Retraso medio entre tardías	10.6 días	Solo 6 534 tardías
Calificación media puntual	4.29 / 5	89 443 reseñas
Calificación media tardía	2.27 / 5	6 381 reseñas

Tabla 2: Línea de base calculada desde la base SQLite del prototipo.

4. Análisis descriptivo y utilidad

La tasa global sirve como referencia para leer las desagregaciones, no como una meta. Entre los estados con al menos 30 pedidos, las tasas más altas son AL (21.4 %; 397 pedidos), MA (17.4 %; 717) y SE (15.2 %; 335). Entre los meses de compra con al menos 30 pedidos, destacan marzo de 2018 (19.0 %; 7 003 pedidos), febrero de 2018 (14.1 %; 6 555) y noviembre de 2017 (12.4 %; 7 288). Estos grupos son candidatos para investigación, no evidencia suficiente para atribuir la demora a una región, un vendedor o una fecha concreta.

En 2018 hay 641 pedidos comprados en feriado nacional, de los cuales 37 fueron tardíos (5.8 %). En los demás días del mismo año hay 52 136 pedidos y 4 041 tardíos (7.8 %). La comparación es desigual en tamaño y no controla por estacionalidad, estado, tipo de producto ni operador logístico; **no se interpreta como efecto causal**. Cuando un grupo filtrado por estado contiene menos de 30 pedidos, el panel oculta su porcentaje de feriados y muestra el tamaño de la muestra.

5. Requerimientos y diseño de la solución

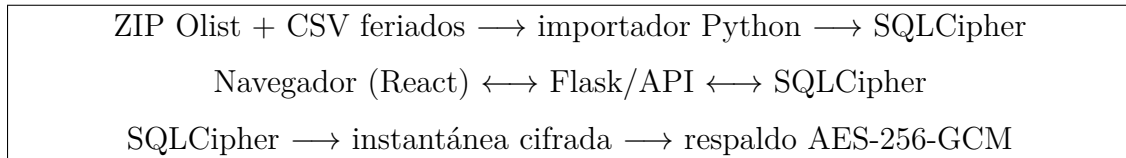
5.1. Requerimientos funcionales

ID	Requerimiento y criterio observable	Estado
F1	Importar las dos fuentes y consultar agregados desde SQLite.	Implementado
F2	Mostrar tasa tardía, retraso y reseñas; filtrar por estado y mostrar serie mensual.	Implementado
F3	Comparar compras en feriado y día común, mostrando tamaños de grupo.	Implementado
F4	Acceder con cuenta y restringir la auditoría al administrador.	Implementado
F5	Consultar registros individuales o análisis por vendedor según rol.	Propuesta futura

Tabla 3: Alcance funcional. “Implementado” se refiere al prototipo local.

5.2. Arquitectura y flujo

El importador Python lee el ZIP Olist y el CSV de feriados, selecciona las columnas necesarias y carga `deliveries` y `holidays` en SQLite cifrada mediante SQLCipher. Flask consulta agregados mediante SQL y expone `/api/dashboard`. React consume esa respuesta y dibuja tarjetas, barras, una tabla mensual y un filtro. Flask conserva las páginas de ingreso y auditoría. La tabla `users` guarda credenciales con hash y rol; `audit_log` registra eventos. El script `backup.py` crea una instantánea cifrada y la protege nuevamente con AES-256-GCM.



El servidor de desarrollo se vincula únicamente a 127.0.0.1. El modo principal usa HTTPS en el puerto 5000 con certificado autofirmado temporal generado por Flask. Existe un modo alternativo HTTP en el puerto 5001 para navegadores que bloquean el certificado; está limitado a la misma máquina y **no constituye protección en tránsito**. No se plantea publicar el servidor de desarrollo en Internet.

5.3. Minimización y ciclo de vida

Por cada entrega se almacenan fechas, estado del cliente, puntuación disponible, identificador de vendedor y variables derivadas. No se importan el identificador de cliente, nombre, dirección, código postal, ciudad ni coordenadas de los CSV originales. El identificador de vendedor se conserva para una extensión futura, pero no se expone en la interfaz actual. El ZIP original, la base, las claves y los respaldos están fuera del control de versiones por `.gitignore`; esto **no cifra ni protege por sí solo** los archivos locales. Antes de compartir el repositorio debe comprobarse su contenido y eliminar datos de prueba no necesarios.

6. Diseño de seguridad y ética

6.1. Modelo de amenazas

Los activos principales son la base operativa, los archivos de origen, los respaldos, las claves, las credenciales y los registros de auditoría. Los adversarios plausibles en la demostración son un usuario no autorizado con acceso al equipo, un visitante de la aplicación local y un operador que comparte por error archivos o capturas. La evaluación de riesgos es cualitativa y se limita al prototipo; no afirma haber realizado una prueba de penetración.

Riesgo	Medida actual	Riesgo residual / medida futura
Lectura indebida de la base o del ZIP	SQLCipher y minimización al importar; rutas fuera de Git	Clave local junto a la base y ZIP sin cifrar; separar claves, restringir permisos y aplicar retención
Robo de credenciales o sesión	Hash de contraseñas, cookie HttpOnly y SameSite, CSRF, límite por usuario	Incorporar MFA y límite por IP; gestionar rotación y caducidad
Intercepción del tráfico	Modo HTTPS local autofirmado	Certificado confiable y servidor de producción; HTTP local no protege transporte
Alteración o pérdida del respaldo	Copia SQLite consistente y AES-256-GCM; prueba de restauración	Separar clave y copia, respaldar fuera del equipo y probar recuperación periódica
Acceso a auditoría o difusión de filas	Control de rol en backend y resultados agregados	Revisar permisos, retención de logs y divulgación accidental de capturas

Tabla 4: Amenazas, controles y límites observables.

6.2. Protección en reposo y en tránsito

Las contraseñas se procesan con las funciones de hash de Werkzeug y no se guardan en texto claro. La creación de cuentas exige un mínimo de 12 caracteres. Flask usa cookies de sesión `HttpOnly` y `SameSite=Lax`; `Secure` está activo en el modo HTTPS y se desactiva solo en el modo HTTP de demostración. Los formularios de ingreso y salida comprueban un token CSRF. Se envían cabeceras `no-store`, `nosniff`, `DENY` para marcos, política de referente y una Content Security Policy restringida al mismo origen.

La base operativa usa SQLCipher 4, con una clave aleatoria de 256 bits aplicada a cada conexión; una cabecera SQLite legible indica que la migración no se ha realizado y bloquea la apertura normal. La clave `instance/db.key` no se versiona y es necesaria para abrir la base. El respaldo crea una instantánea cifrada y la protege adicionalmente con AES-256-GCM, *nonce* aleatorio de 12 bytes y autenticación del formato. La restauración se comprueba con `PRAGMA integrity_check`. **Limitación importante:** la clave de SQLCipher y la de respaldo siguen en el mismo equipo y el ZIP original no está cifrado. Cifrar el archivo de base protege ante una copia aislada de ese archivo, pero no ante el compromiso del equipo completo. Para un entorno real se requerirían custodia separada de claves, permisos de archivo, cifrado del ZIP y rotación documentada.

El certificado autofirmado local demuestra el uso técnico de HTTPS, pero un navegador no puede validarlo como identidad de confianza sin instalar una CA propia. No se instruye a ignorar advertencias de seguridad como práctica normal. Para producción se necesitaría una CA confiable y un servidor apropiado. No se afirma cumplimiento de un estándar por el mero uso de estas funciones.

6.3. Accesos y registros de auditoría

El backend reconoce los roles `admin`, `analyst` y `reader`. Los tres pueden ver el panel agregado; solo `admin` puede consultar `/audit`. No existe una interfaz de administración

de usuarios ni privilegios analíticos diferenciados para los otros dos roles. Se bloquea temporalmente el ingreso después de cinco intentos fallidos por usuario en 15 minutos; aún falta un límite complementario por origen y una política para evitar bloqueos deliberados de cuentas.

El registro almacena fecha UTC, nombre de usuario y acción (ingreso, fallo, bloqueo, salida o consulta), sin registrar contraseñas. La página administrativa presenta los últimos 100 eventos; ello **no significa que la tabla conserve solo 100**. No se ha implementado retención ni borrado automático de logs. Una política de producción deberá fijar acceso, plazo de conservación, exportación y revisión periódica.

6.4. Protección de datos y marco de referencia

El hecho de que Olist publique los datos no autoriza cualquier uso posterior. Se adopta la minimización y se muestran resultados agregados para evitar republicar registros y ubicaciones exactas. Si el prototipo llegara a tratar datos personales reales en Perú, el responsable tendría que evaluar la Ley N.º 29733 y su reglamento aprobado por D. S. N.º 016-2024-JUS, entre otras obligaciones aplicables [4]. La mención de estas normas no equivale a una declaración de cumplimiento legal ni resuelve por sí sola la posible aplicación de normas brasileñas. Como referencia de gestión se utiliza el ciclo gobernar, identificar, proteger, detectar, responder y recuperar del NIST CSF 2.0 [5].

7. Estrategia de uso seguro, políticas y formación

7.1. Política propuesta para el equipo

Esta política se formula para la demostración y deberá ser aceptada y aplicada por ambos integrantes antes de compartir el material. Su cumplimiento organizativo todavía no se ha auditado.

1. **Finalidad:** usar los datos solo para el proyecto académico y análisis agregado de entregas. Prohibir decisiones individuales sobre compradores o vendedores basadas en este panel.
2. **Acceso:** crear una cuenta individual por integrante y otorgar el rol mínimo necesario. No compartir contraseñas ni sesiones; cerrar sesión al terminar la demostración.
3. **Almacenamiento y publicación:** no subir ZIP, SQLite, claves, respaldos ni capturas con filas individuales a GitHub, Notion o servicios públicos. Revisar `git status` y el contenido antes de publicar.
4. **Transferencia:** compartir solo el PDF y el código necesario; para una demostración remota usar HTTPS con certificado confiable. El HTTP en 127.0.0.1 solo se permite en el mismo equipo.
5. **Retención:** conservar los archivos locales únicamente durante el curso o el plazo acordado con el profesor; al cierre, eliminar copias y verificar que el repositorio no contenga secretos.

6. **Uso analítico:** mostrar denominadores y advertir que correlación, feriados y diferencias geográficas no prueban causas.

7.2. Procedimiento y concientización

Antes de cada presentación, ambos integrantes deben revisar el modo de servidor, la cuenta utilizada, las rutas del repositorio y las capturas previstas. Si se descubre un secreto en un commit, debe dejar de distribuirse ese repositorio y rotarse el secreto; borrar solo el archivo actual no elimina su historial. Se propone una sesión breve de capacitación entre integrantes con cuatro ejercicios: reconocer un dato potencialmente identificable, distinguir HTTP local de HTTPS validado, restaurar un respaldo de prueba y ensayar cómo reportar una exposición accidental. La sesión y sus asistentes deben registrarse cuando efectivamente se realice; **no se presenta como una capacitación ya ejecutada.**

8. Respuesta ante incidentes y recuperación

8.1. Plan básico

Se consideran incidentes la pérdida de la base, publicación accidental de ZIP o claves, acceso no autorizado y alteración de resultados. El plan siguiente es un procedimiento propuesto; no sustituye asesoría especializada ni fija plazos legales sin evaluar la jurisdicción aplicable. La secuencia está alineada de manera general con las recomendaciones de respuesta de NIST SP 800-61 Rev. 3 [6].

Fase	Acción del equipo	Evidencia / resultado esperado
Detección	Quien observe el incidente avisa de inmediato al otro integrante; ambos registran hora UTC, sistema afectado y señal observada.	Ficha de incidente sin copiar secretos al informe.
Contención	Detener el servicio si procede, aislar el equipo, retirar publicaciones accidentales y revocar credenciales comprometidas.	Registro de acciones y hora; preservar una copia para análisis.
Análisis	Revisar logs, repositorio y alcance de archivos expuestos; distinguir pérdida, acceso y divulgación.	Alcance provisional e incertidumbres explícitas.
Recuperación	Restaurar desde un respaldo cifrado conocido, verificar integridad y número de registros, renovar claves y validar el panel.	<code>integrity_check = ok</code> y consulta de control.
Comunicación	Informar al profesor y a los responsables definidos por el equipo; evaluar con asesoría las obligaciones legales si hubiera datos personales.	Comunicación precisa, sin afirmar ausencia de daño antes de investigar.
Mejora	Documentar causa, correcciones y nueva prueba; actualizar el procedimiento.	Lecciones aprendidas y responsable de seguimiento.

Tabla 5: Procedimiento propuesto para incidentes del prototipo.

8.2. Contingencia y respaldo

La copia cifrada se genera con `py backup.py backup` y se restaura a una ruta distinta con `py backup.py restore`. La restauración de prueba produjo `integrity_check = ok` y una base con 96 470 entregas. Esto verifica que *esa copia de prueba* era recuperable; no prueba resistencia a una pérdida simultánea del equipo y de la clave. En una operación real se programarían copias periódicas, almacenamiento separado, inventario de claves, pruebas regulares y objetivos RPO/RTO definidos con el negocio.

9. Implementación y validación

9.1. Componentes desarrollados

El importador `import_data.py` integra Olist y los feriados; `db_crypto.py` aplica la clave SQLCipher a las conexiones. `app.py` ofrece autenticación, API y auditoría; `frontend/src/App.tsx` muestra el panel. `backup.py` genera y restaura respaldos cifrados.

9.2. Pruebas y evidencias técnicas

Comprobación	Resultado observado	Evidencia reproducible
Importación	96 470 filas incluidas; 2 971 excluidas.	Conteo SQLite e importador
Cifrado en reposo	La base migrada no tiene cabecera SQLite legible; SQLCipher la abre con clave y conserva 96 470 filas.	<code>migrate_db.py</code> y consulta cifrada
Acceso	La ruta principal exige sesión; la auditoría devuelve 403 a un lector.	Rutas Flask / rol
Validación de filtro	Estado válido actualiza cifras; formato inválido responde 400.	API y navegador
Intentos fallidos	Tras cinco fallos recientes por usuario, se responde 429.	Lógica y prueba local
Recuperación	Copia cifrada restaurada; integridad ok, 96 470 filas.	<code>backup.py</code> y SQLite
Frontend	Compilación TypeScript/Vite completada y filtro observado en navegador.	<code>npm run build</code>
Auditoría	Solo el administrador ve fecha UTC, usuario y acción.	Ruta <code>/audit</code>

Tabla 6: Pruebas del prototipo local. No equivalen a una auditoría externa.

No se han realizado pruebas de carga, revisión externa de seguridad ni validación de un despliegue de producción. La existencia del modo HTTPS se verifica por la configuración del servidor y la negociación TLS local; una evidencia de transporte plenamente confiable requeriría un certificado validado por una CA pública y un entorno de producción formal.

9.3. Evidencias visuales del prototipo

A continuación se presentan las capturas del prototipo en funcionamiento, documentando la interfaz del panel general, la reactividad del filtrado regional, el control de acceso a la auditoría y las pruebas de seguridad ejecutadas sobre la base de datos y el transporte.

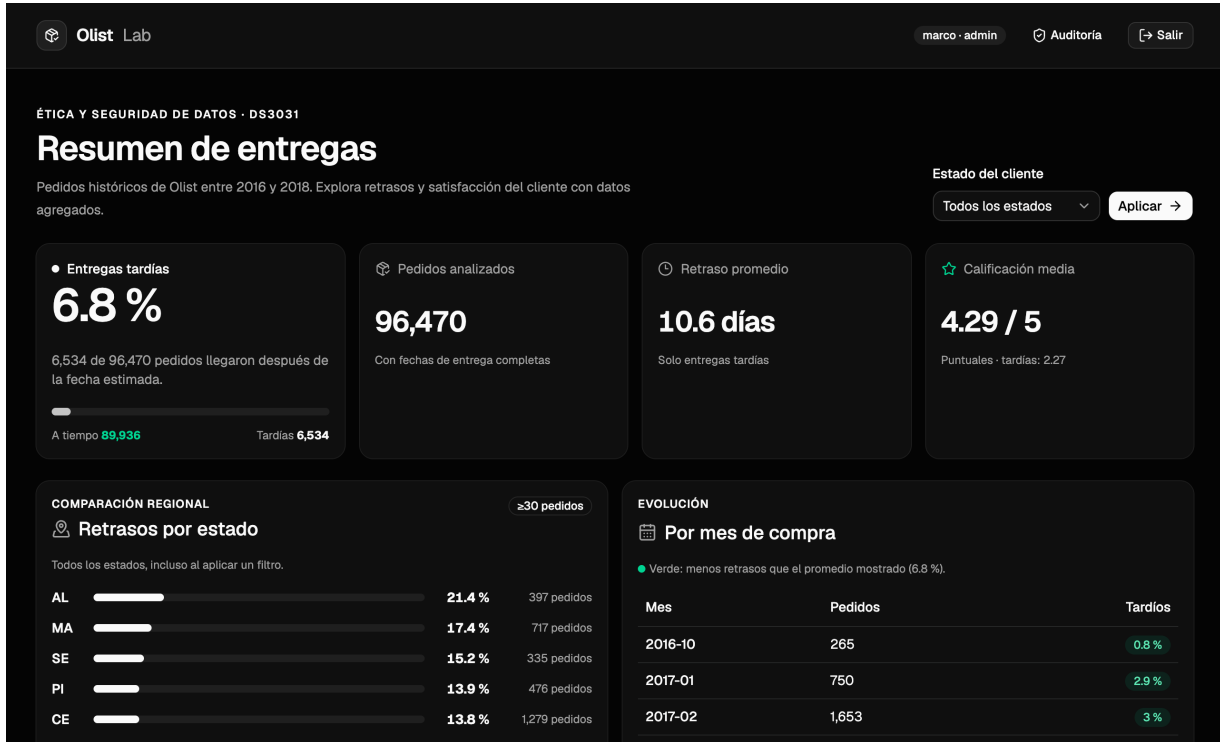


Figura 1: Panel general: cuatro indicadores principales y detalle de fuentes.

Fuente: Captura del prototipo local en ejecución (React + Tailwind + Flask).

Detalle y validación: La interfaz principal del panel analítico muestra la sesión del usuario administrador (marco · admin) y los cuatro indicadores calculados sobre los 96 470 pedidos entregados y depurados:

- *Entregas tardías:* 6.8 % de tasa global (6 534 pedidos entregados con demora frente a 89 936 puntuales).
- *Pedidos analizados:* 96 470 registros con fechas completas.
- *Retraso promedio:* 10.6 días entre pedidos tardíos.
- *Calificación media:* 4.29/5 en pedidos puntuales y 2.27/5 en pedidos con retraso.

En la parte inferior se observa la distribución por estados (destacando AL con 21.4 % y MA con 17.4 %) y la evolución por mes de compra. El pie de página documenta las fuentes empleadas (dataset Olist y Portaria n.º 468/2017). Esta vista confirma que los datos se presentan agregados, sin exponer nombres, direcciones ni identificadores de clientes.

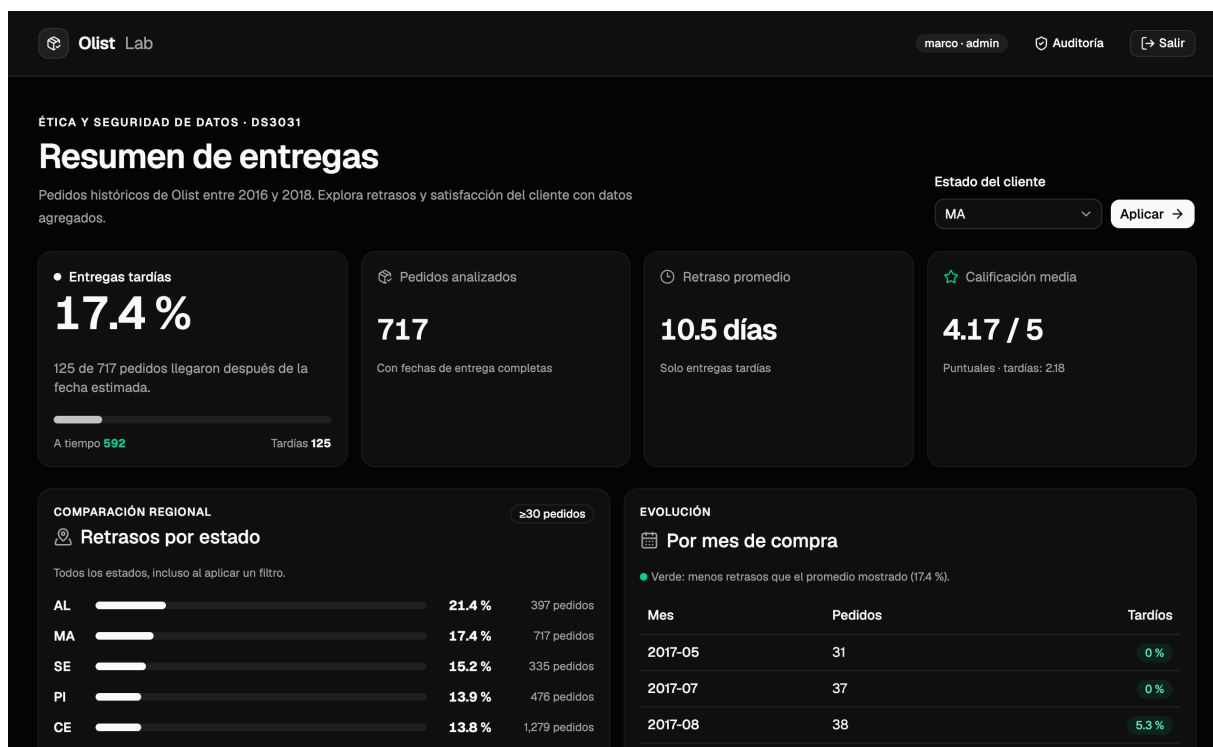


Figura 2: Filtro por estado federado: vista filtrada para Maranhão (MA).

Fuente: Captura del prototipo filtrado por parámetro geográfico (?state=MA).

Detalle y validación: Al aplicar el filtro para el estado de Maranhão (MA), la aplicación actualiza las cifras mediante la API: el total de pedidos analizados pasa a 717, la tasa de tardanza sube a 17.4 % (125 entregas tardías), el retraso medio se sitúa en 10.5 días y la satisfacción desciende a 4.17/5 (puntuales) y 2.18/5 (tardíos). La tabla mensual inferior se ajusta automáticamente para mostrar únicamente la serie temporal de este estado, verificando la consistencia de las consultas parametrizadas en el backend SQL sin riesgo de inyecciones.



Figura 3: Control de acceso basado en roles (RBAC) y registro de auditoría.

Fuente: Capturas del comportamiento del servidor según el rol de sesión.

Detalle y validación: En el panel superior (a), la ruta administrativa /audit es accesi-

ble para el rol **admin** (marco), listando en orden cronológico inverso los eventos de auditoría: marca de tiempo UTC, usuario y acción (**login**, **dashboard_api**, **dashboard_api:MA**, **audit_view**). En el panel inferior (b), se evidencia la restricción de privilegios: cuando un usuario con rol **reader** (lector) intenta ingresar a dicha ruta, Flask bloquea la solicitud y devuelve un código **HTTP 403 Forbidden**, impidiendo el acceso a los registros de auditoría a roles no autorizados.

```
marcosoto@MacBook-Pro: ~/Desktop/proyecto-etica-seguridad-olist - zsh

[PRUEBA 1] Verificación de Cifrado en Reposo (SQLCipher vs SQLite Estándar)
→ olist-lab (main) xxd -l 16 instance/olist.db
00000000: cd25 1341 1404 9ed7 d0b0 bef7 75eb f460 .%.A.....U..`
# Cabecera binaria cifrada sin el prefijo estándar "SQLite format 3"
→ olist-lab (main) sqlite3 instance/olist.db "SELECT count(*) FROM deliveries;"
Error: in prepare, file is not a database (26)
# Intento de lectura sin clave criptográfica: bloqueado por ausencia de clave
→ olist-lab (main) .venv/bin/python -c "from db_crypto import connect; print('Conteo:', connect().execute('SELECT
count(*) FROM deliveries').fetchone()[0])"
✓ Apertura con SQLCipher y clave local (db.key): Conteo: 96470 pedidos

[PRUEBA 2] Contingencia y Restauración Íntegra (AES-256-GCM)
→ olist-lab (main) .venv/bin/python backup.py backup instance/copia.enc
Respaldo cifrado generado: instance/copia.enc (AES-256-GCM con nonce aleatorio de 12 bytes)
→ olist-lab (main) .venv/bin/python backup.py restore instance/copia.enc --destination instance/restaurada.db
✓ Restauración y verificación completadas: instance/restaurada.db (PRAGMA integrity_check = ok, 96,470
registros)

[PRUEBA 3] Transporte Seguro en Tránsito (HTTPS Local con TLS 1.3)
→ olist-lab (main) curl -k -v -I https://127.0.0.1:5002/login 2>&1 | grep -E "(TLS handshake|SSL connection|Server
certificate|HTTP/1.1|subject:)"
* (304) (IN), TLS handshake, Server hello (2)
* (304) (IN), TLS handshake, Certificate (11)
* SSL connection using TLSv1.3 / AEAD-AES256-GCM-SHA384
* Server certificate: subject: O=Dummy Certificate; CN=* (Autofirmado ad-hoc)
< HTTP/1.1 200 OK (Sesión protegida con cabecera Cache-Control: no-store y cookies Secure)
```

Figura 4: Pruebas técnicas de seguridad: cifrado en reposo, contingencia y HTTPS.

Fuente: Captura de consola ejecutando pruebas de verificación.

Detalle y validación: En la sesión de consola se comprueban tres mecanismos de seguridad implementados:

1. *Cifrado en reposo (SQLCipher):* La lectura inicial de los primeros 16 bytes de `instance/olist.db` mediante `xxd` muestra bytes aleatorios y confirma la ausencia de la cadena legible `SQLite format 3`. El cliente estándar `sqlite3` rechaza el archivo con `Error: file is not a database (26)`, mientras que la conexión mediante `SQLCipher` con la clave local de 256 bits (`instance/db.key`) abre la base y recupera los 96 470 registros.
2. *Contingencia y recuperación:* Ejecución de `backup.pybackup`, generando un respaldo cifrado y autenticado con `AES-256-GCM` (`instance/copia.enc`). Su posterior restauración con `backup.pyrestore` comprueba la integridad estructural (`PRAGMA integrity_check = ok`) y confirma la recuperación exacta de las 96 470 filas en una base independiente.

3. *Protección en tránsito (HTTPS local)*: Negociación TLS 1.3 con cifrado AES-256-GCM y certificado autofirmado ad-hoc en Flask, validando la entrega de cabeceras de seguridad (`Cache-Control: no-store` y cookies con atributo `Secure`).

10. Recomendaciones de protección futura

1. **Separar y proteger las claves**: SQLCipher cifra la base operativa, pero su clave se guarda localmente; usar un gestor de secretos o el almacén protegido del sistema y documentar rotación y recuperación.
2. **Desplegar con TLS de CA confiable**: elimina la advertencia propia del certificado temporal y permite protección en tránsito verificable para usuarios remotos. No exponer el modo HTTP local.
3. **Fortalecer identidad y sesiones**: añadir MFA, limitación por IP, recuperación de cuentas, expiración de sesión y rotación de credenciales para reducir abuso de cuentas.
4. **Operar respaldos fuera del equipo**: programar copias, separar la clave, verificar restauración regularmente y definir RPO/RTO con el negocio para resistir fallos del dispositivo.
5. **Gestionar logs y retención**: limitar acceso administrativo, conservar solo el plazo necesario, revisar eventos y eliminar registros vencidos; hoy la tabla crece sin limpieza automática.
6. **Revisar privacidad y calidad**: comprobar condiciones de uso del dataset, duplicados de reseñas, valores faltantes y sesgos antes de ampliar el panel o usar datos reales.

Estas medidas se recomiendan porque reducen riesgos concretos identificados; no se declaran implementadas.

11. Lecciones aprendidas y retrospectiva del equipo

11.1. Lecciones técnicas y analíticas

La principal lección técnica es que un dataset público puede contener columnas que no se necesitan para responder la pregunta de negocio. Definir primero el denominador del KPI permitió importar menos campos y detectar los 646 pedidos sin reseña. La comparación con feriados mostró que una cifra atractiva puede ser engañosa cuando los tamaños de grupo son muy distintos. En seguridad, restaurar una copia aportó evidencia más fuerte que limitarse a generar el archivo cifrado; al mismo tiempo, dejó visible la debilidad de guardar clave y base en el mismo equipo.

11.2. Retrospectiva del trabajo en equipo y mejoras futuras

Este proyecto lo desarrollamos Ronal Jesus Condor Blas y Marco Soto Maceda. Basamos esta retrospectiva en el trabajo documentado en el repositorio, sin atribuir porcentajes de participación ni tareas individuales que no hayamos registrado. Para mejorar nuestro trabajo conjunto en proyectos futuros, proponemos:

1. Llevar un registro estructurado de decisiones técnicas y de arquitectura para mantener la trazabilidad de cada cambio.
2. Establecer un flujo sistemático de revisión cruzada de código (*peer review*) antes de integrar cambios en el importador y en los controles de acceso.
3. Ejecutar pruebas conjuntas periódicas para verificar la restauración de respaldos y el comportamiento ante incidentes.
4. Formalizar una lista de verificación previa a la entrega para constatar la exclusión de claves y la vigencia de la documentación técnica.

El código fuente y la versión interactiva del informe se encuentran disponibles en el repositorio del proyecto (<https://github.com/ronalforge/proyecto-etica-seguridad-olist>) y en su correspondiente despliegue en GitHub Pages.

12. Conclusiones

El prototipo responde al caso de negocio con métricas históricas verificables y una interfaz agregada. La línea de base de 6.8 % de entregas tardías y la diferencia de puntuaciones permiten priorizar preguntas para una investigación logística posterior, pero no establecen causas ni metas operativas actuales. La integración de feriados añade una segunda fuente y demuestra por qué los grupos pequeños y las comparaciones no controladas requieren cautela.

En materia de seguridad, la solución implementa controles visibles de autenticación, roles, auditoría, base operativa cifrada con SQLCipher, HTTPS local y respaldo cifrado con restauración. La evaluación también identifica límites sustanciales: el ZIP original no está cifrado, las claves siguen en el mismo equipo, el certificado es autofirmado y el modo HTTP local no ofrece confidencialidad de transporte. El informe distingue estas condiciones de la hoja de ruta futura. Por ello, el resultado es una **demostración académica funcional**, no un sistema listo para operar con datos personales reales en producción.

Referencias

- [1] Curso DS3031. *Esquema Proyecto de Ética y Seguridad de Datos – Tipo 1*. Material de clase proporcionado por el profesor.
- [2] Olist. *Brazilian E-Commerce Public Dataset by Olist*. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. Consulta: septiembre de 2026.

- [3] Ministério do Planejamento, Desenvolvimento e Gestão de Brasil. *Portaria n.º 468, de 22 de dezembro de 2017*. <https://www.gov.br/agricultura/pt-br/mailling/documentos/portaria-mpog-feriados-2018.pdf>. Consulta: septiembre de 2026.
- [4] Autoridad Nacional de Protección de Datos Personales. *Decreto Supremo N.º 016-2024-JUS: Reglamento de la Ley N.º 29733*. <https://www.gob.pe/institucion/anpd/normas-legales/6554453-n-016-2024-jus>. Consulta: septiembre de 2026.
- [5] National Institute of Standards and Technology. *The NIST Cybersecurity Framework (CSF) 2.0*, 2024. <https://www.nist.gov/cyberframework>. Consulta: septiembre de 2026.
- [6] National Institute of Standards and Technology. *SP 800-61 Rev. 3: Incident Response Recommendations and Considerations for Cybersecurity Risk Management*, 2025. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>. Consulta: septiembre de 2026.